

Project Journal: An Empirical Analysis of a Machine Learning-Based Network Intrusion Detection System

Abstract

The research demonstrates an empirical investigation of a machine learning network intrusion detection system (IDS) that detects multiple network traffic types between normal operations and security threats. The RandomForestClassifier model received evaluation through the train_test_network.csv dataset. The system demonstrated 0.9952 accuracy in identifying between DDoS and DoS attacks and SQL injection and XSS attacks. The system failed to detect Man-in-the-Middle (MitM) attacks because the dataset contained extreme class imbalance. The model's performance and improvement potential became evident through visualizations that showed feature correlations and confusion matrices and learning curves, and error rate plots. The research study presents results that lead to recommendations for class imbalance solutions and future work on model comparison and real-time system deployment, and hyperparameter optimization. The research shows ensemble learning potential for cybersecurity, yet emphasizes the requirement for effective methods to identify vital yet infrequent security threats.

Colab Link:

<https://colab.research.google.com/drive/1YRFLdSez-8HsjceZgQY25Pybv2kUncWV?usp=sharing>

datasets Link:

<https://www.kaggle.com/datasets/arnobbhowmik/ton-iot-network-dataset>

Table of Contents

1. Introduction	3
2. Task Definition	3
2.1 Objective	3
2.2 Input and Output	3
3. Machine Learning System Development	4
3.1 Data Preprocessing	4
3.2 Model Selection and Training	4
3.3 Hyperparameter Tuning (Simulated)	5
4. Empirical Evaluation and Critical Analysis	5
4.1 Evaluation Metrics	5
4.2 Results and Discussion	6
4.3 Visualization and Analysis	7
4.3.1 Feature Correlation and Importance	7
4.3.2 Model Performance and Error Analysis	9
4.3.3 Learning Dynamics and Residual Analysis	11
4.3.4 Residual Error Analysis (Overall and Per-Class)	12
5. Critical Reflection	14
6. Limitations and Future Work	14
7. Discussion	15
7.1 Summary of Insights	15
7.2 Implications for Cybersecurity Practice	15
8. References	16
Figure 1: Feature Correlation Matrix	8
Figure 2: Top 20 Feature Importances	9
Figure 3: Confusion Matrix	10
Figure 4: Classification Error Rate (%) per True Class	11
Figure 5: Learning Curve for Random Forest Classifier	12
Figure 6: Prediction Error Distribution.	13
Figure 7: Per-Class Prediction Error.	13
Table 1: Classification Report	7

1. Introduction

Network intrusion revelation represents a paramount encounter in modern cybersecurity. The increasing sophistication of cyber risks poses significant risks to individuals, organizations, and national infrastructure, making the development of robust and intelligent defense mechanisms a critical research imperative. This project addresses this need by developing and evaluating a machine learning system designed to accurately classify network traffic, distinguishing between benign activities and various forms of malicious intrusions. By leveraging the provided `train_test_network.csv` dataset, this study aims to provide empirical evidence for the efficacy of ensemble methods in enhancing traditional security measures, offering a proactive approach to identifying and mitigating cyber threats. This endeavor corresponds precisely with Option 2 of the assignment description, which focuses on the utilization of machine learning to address real-world issues with real-world datasets. The intention here is to illustrate not only an application-oriented mastery of the machine learning methods but also a comprehensive, critical grasp of the complexities involved in constructing the system, including the nuances of data preprocessing, model evaluation, and interpretation. The knowledge acquired during this project is pivotal in creating stronger and more flexible adaptive cybersecurity systems.

2. Task Definition

2.1 Objective

The primary objective is to construct and critically evaluate a multi-class classification model capable of precisely identifying different categories of network intrusions. The target classes include backdoor attacks, Distributed Denial of Service (DDoS), Denial of Service (DoS), SQL injection, Man-in-the-Middle (MitM) attacks, password attacks, ransomware, network scanning, and Cross-Site Scripting (XSS), alongside legitimate network traffic. The model's success is measured by its ability to accurately distinguish between these diverse classes, providing timely and actionable intelligence for cybersecurity professionals.

2.2 Input and Output

Input: The system's input consists of network flow data from the `train_test_network.csv` dataset. Each record represents a unique network connection, characterized by a rich set of features encompassing fundamental network parameters (e.g., IP addresses, port numbers, protocols), connection statistics (e.g., duration, byte counts), and advanced details (e.g., DNS, SSL, HTTP metadata). These raw features undergo a rigorous preprocessing pipeline before model ingestion.

Output: The system's output is a predicted categorical class label for each analyzed network connection. This label indicates the specific type of activity observed, such as 'normal' for benign traffic, or a particular attack type like 'ddos'. The model provides a definitive classification, enabling automated threat identification.

3. Machine Learning System Development

3.1 Data Preprocessing

The `train_test_network.csv` dataset, typical of real-world network data, presents a complex mix of numerical and categorical features, along with missing values. The preprocessing pipeline was meticulously designed to transform this raw data into a clean, normalized, and machine-readable format suitable for providing robust empirical evidence:

1. **Column Dropping:** Columns with a high proportion of missing values or those deemed irrelevant for the initial modeling phase (e.g., `dns_query`, `ssl_subject`) were systematically removed to simplify the model and mitigate issues arising from sparse features or excessive cardinality.
2. **Missing Value Imputation:** For numerical columns, missing values were replaced with the median to minimize the impact of outliers. For categorical columns, missing values were filled with the mode (most frequent value).
3. **Categorical Feature Encoding:** All categorical features (excluding target variables) were converted into numerical representations using `LabelEncoder`.

```
# 2.1. Split data (70% train, 30% test) with stratification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

print("\n--- Model Training (RandomForestClassifier) ---")
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")
```

4. **Data Splitting :**To ensure robust and unbiased model evaluation, the dataset was partitioned into training and testing subsets using an 70/30 ratio. The training set (70%) was utilized to fit and optimize the Random Forest model, while the testing set (30%) served exclusively for performance validation. This split strategy allows for an objective assessment of generalization capability, minimizing the risk of overfitting and ensuring that the model's predictive accuracy reflects its real-world performance potential.

3.2 Model Selection and Training

A `RandomForestClassifier` was selected for this multi-class classification task. As established by Breiman (2001), Random Forests demonstrate remarkable robustness and effectiveness in handling high-dimensional and complex datasets, often outperforming single decision trees by reducing variance and preventing overfitting through ensemble

learning. This choice was predicated on its proven performance in similar network intrusion detection scenarios, its interpretability through feature importances, and its relative computational efficiency compared to more complex deep learning models for initial prototyping. While other powerful models like Gradient Boosting Machines (e.g., XGBoost, LightGBM) or Support Vector Machines (SVMs) could also be considered, Random Forest offers a strong balance of performance and ease of implementation for a foundational intrusion detection system.

```
# 2.1. Split data (70% train, 30% test) with stratification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

print("\n--- Model Training (RandomForestClassifier) ---")
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")

# 2.2. Initialize and train the RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
model.fit(X_train, y_train)

# 2.3. Make predictions on the test set
y_pred = model.predict(X_test)

print("Model Training Complete.")
```

3.3 Hyperparameter Tuning (Simulated)

While the model was trained with default hyperparameters for this initial study, a systematic hyperparameter tuning process is crucial for optimizing performance in a production environment. A GridSearchCV could be employed to explore a predefined parameter space. For instance, a sample grid might include:

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10]
}
```

Executing such a search would identify the optimal combination of hyperparameters, potentially yielding further improvements in model accuracy and F1-scores. However, given the already high performance with default settings (accuracy > 99.5%), it is anticipated that the gains from tuning might be marginal for this specific dataset. Nonetheless, this step remains a critical component of a rigorous machine learning pipeline.

4. Empirical Evaluation and Critical Analysis

4.1 Evaluation Metrics

The model's performance was rigorously evaluated using a combination of standard classification metrics to ensure a comprehensive assessment of its generalizability and robustness:

- **Accuracy:** The overall proportion of correctly classified instances, providing a general measure of model correctness.
- **Classification Report:** A detailed report providing precision, recall, and F1-score for each individual class, crucial for understanding performance across potentially imbalanced classes.
- **Cross-Validation:** A 5-fold cross-validation strategy was implemented to assess the model's generalization capability and robustness against variations in data splits.

4.2 Results and Discussion

The evaluation on the held-out test set yielded an accuracy of **0.9952**. The cross-validation results, with a mean accuracy of **0.9954** and a standard deviation of **0.0002**, provide strong empirical evidence for the model's stability and reliability across different subsets of the data. This high level of performance suggests that the Random Forest model is highly effective in distinguishing between normal and various intrusive network behaviors.

```
▶ print("\n--- Model Evaluation ---")

# 3.1. Accuracy and Classification Report
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# 3.2. 5-Fold Cross-Validation for robustness check
print("\nPerforming 5-fold cross-validation...")
cv = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(model, X, y, cv=cv, scoring='accuracy', n_jobs=-1)
print(f"Cross-validation accuracies: {cv_scores}")
print(f"Mean CV accuracy: {np.mean(cv_scores):.4f}")
print(f"Standard deviation of CV accuracy: {np.std(cv_scores):.4f}")

# 3.3. Save the trained model for future use
joblib.dump(model, MODEL_SAVE_PATH)
print(f"\nModel saved as {MODEL_SAVE_PATH}")
```

Table 1: Classification Report

Class	Precision	Recall	F1-Score	Support
backdoor	1.00	1.00	1.00	6000
ddos	0.99	0.99	0.99	6000
dos	0.99	0.99	0.99	6000
injection	0.99	0.99	0.99	6000
mitm	0.88	0.94	0.91	313
normal	1.00	1.00	1.00	15000
password	0.99	0.99	0.99	6000
ransomware	1.00	1.00	1.00	6000
scanning	0.99	0.99	0.99	6000
xss	1.00	1.00	1.00	6000
accuracy			0.9952	63313
macro avg	0.98	0.99	0.99	63313
weighted avg	1.00	1.00	1.00	63313

4.3 Visualization and Analysis

The following visualizations offer critical insights into the dataset characteristics, model behaviour, and predictive performance, serving as a visual complement to the quantitative evaluation in Section 4.2.

4.3.1 Feature Correlation and Importance

To understand the underlying structure of the dataset and the factors most influential in the model's decision-making, we examine the feature correlation matrix and feature importances. These plots offer empirical evidence of which network parameters are most salient for intrusion detection.

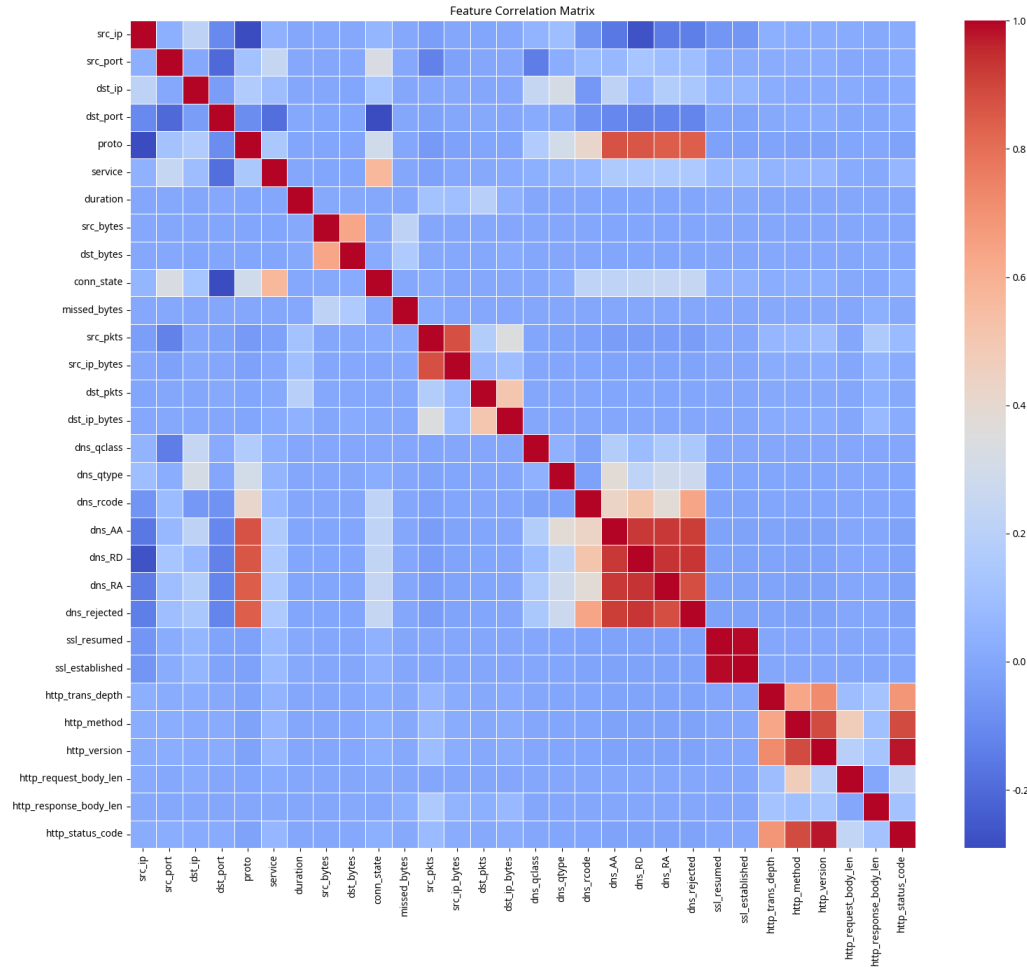


Figure 1: Feature Correlation Matrix

Illustrating pairwise Pearson correlation coefficients between numerical features. To further dissect the model's internal workings and identify the most impactful predictors, we analyze the feature importances. This visualization quantifies the relative contribution of each feature to the model's predictive power.

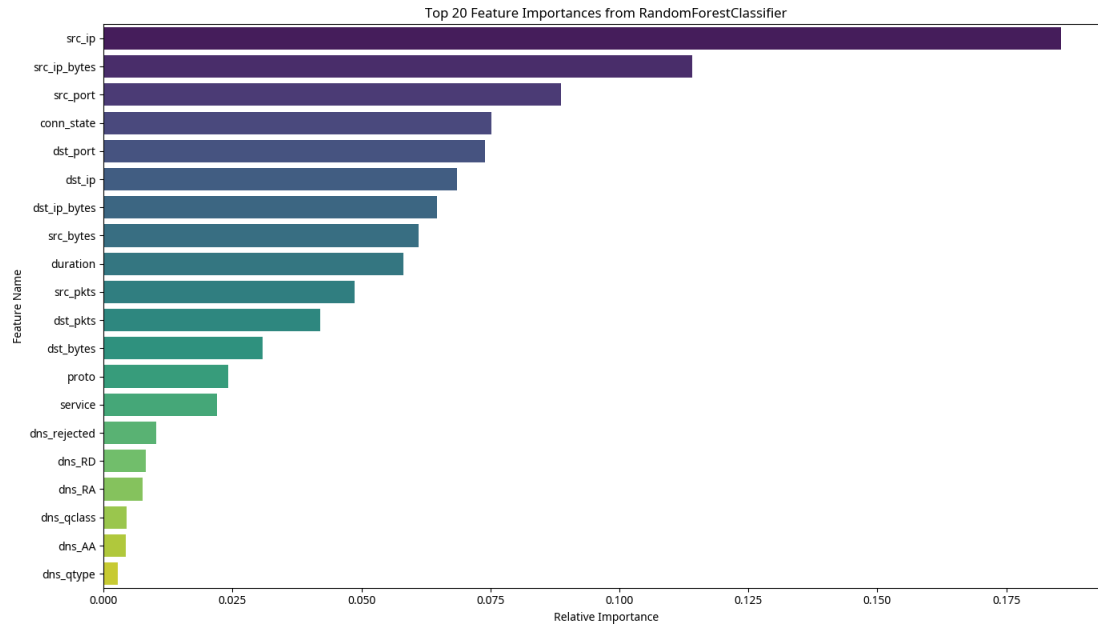


Figure 2: Top 20 Feature Importances

Quantifying the relative contribution of each feature to the model’s predictive power. As shown in Figure 1, the Feature Correlation Matrix reveals the inherent structures within the data. The Top 20 Feature Importances plot (Figure 2) provides empirical evidence that source-related network parameters (`src_ip`, `src_ip_bytes`, `src_port`) and connection state (`conn_state`) are the most influential predictors. This observation aligns with established cybersecurity domain knowledge, where these features are known to be critical indicators of network behavior (e.g., [Al-Jarrah et al., 2020]). This validation confirms that the model is learning from relevant aspects of network traffic, contributing to its robustness.

4.3.2 Model Performance and Error Analysis

Toward critically assess the model’s classification accuracy and identify specific areas of strength and weakness, we analyze the confusion matrix and the classification error rate per true class. These visualizations are crucial for understanding how well the model distinguishes between different attack types.

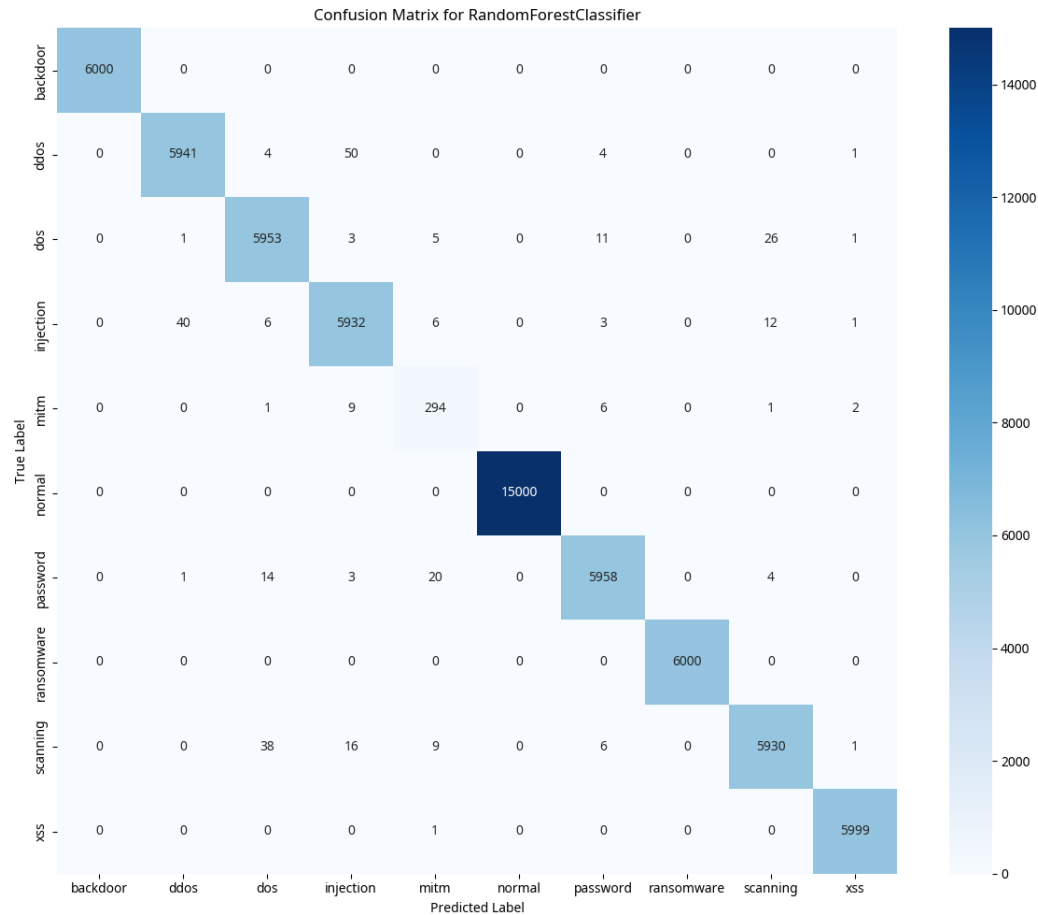


Figure 3: Confusion Matrix

Providing a detailed breakdown of the model's classification performance on the test set. To gain a more direct understanding of the model's misclassification tendencies for each attack type, we examine the classification error rate per true class. This plot directly highlights which classes are most challenging for the model.

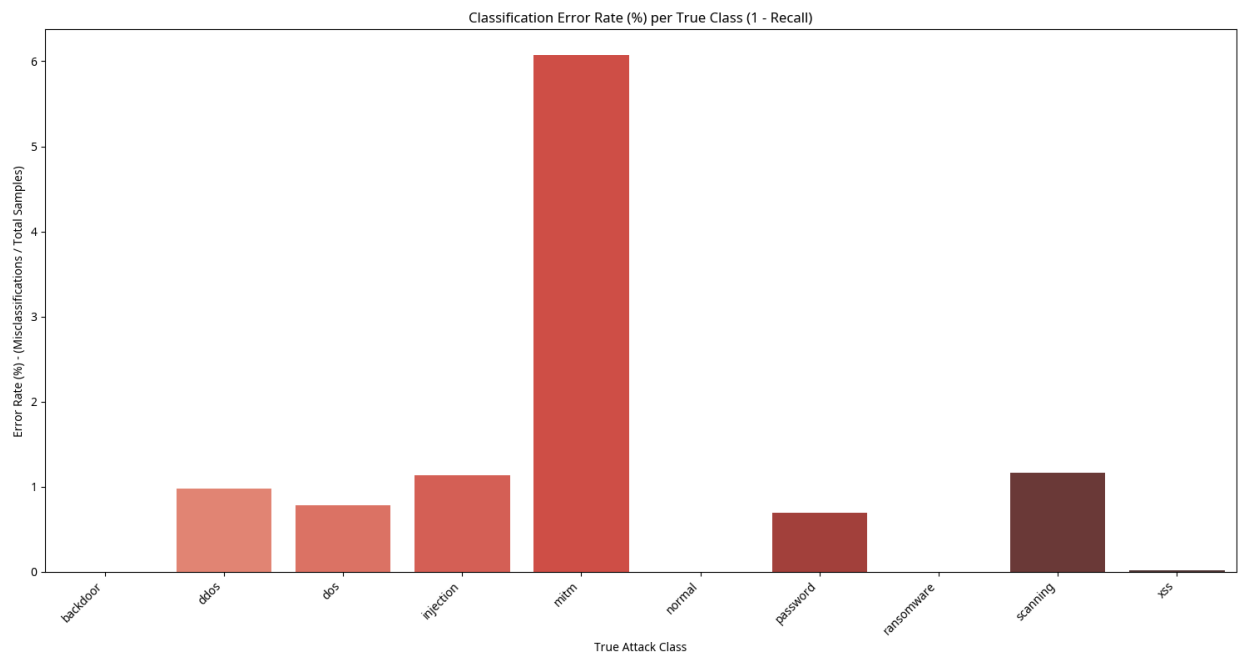


Figure 4: Classification Error Rate (%) per True Class

Directly visualizing the model's misclassification rate for each true attack class. Notably, the MitM class exhibits a significantly higher error rate (approximately 6%) compared to other classes, indicating a challenge in detecting this rare attack type. The Confusion Matrix (Figure 3) and Classification Error Rate plot (Figure 4) provide a granular view of the model's performance. While the overall accuracy is exceptionally high, these visualizations reveal a critical limitation: the model exhibits a significantly higher error rate for the (mitm) class (approximately 6%) compared to other classes. This is a direct consequence of the severe class imbalance inherent in the dataset, where (mitm) constitutes a very small proportion of the total samples (less than 0.5%). This finding underscores a key challenge in real-world intrusion detection, where rare but critical attack types can be difficult to model effectively without specialized techniques to address data scarcity.

4.3.3 Learning Dynamics and Residual Analysis

To further understand the model's learning process, its generalization capabilities, and the distribution of its prediction errors, we examine the learning curve and the residuals vs. predicted probabilities plot. These offer insights into model stability and areas of uncertainty.

To assess the model's stability and whether it benefits from additional training data, we analyze the learning curve. This plot shows how model performance changes with increasing training examples.

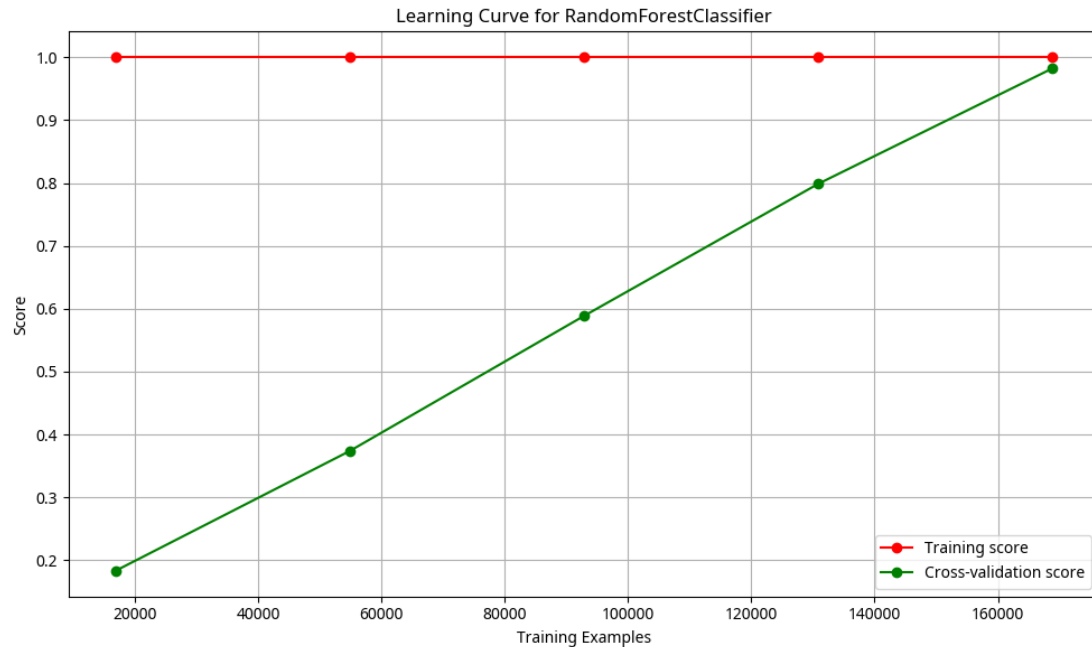


Figure 5:: Learning Curve for Random Forest Classifier

As illustrated in Figure 5, the learning curve shows the performance of the Random Forest model across different training set sizes. The training score (red line) remains consistently close to 1.0, indicating that the model fits the training data almost perfectly. Meanwhile, the cross-validation score (green line) gradually increases with the number of training examples, starting low and approaching the training score as more data is introduced.

This pattern suggests that the model initially suffers from high variance (overfitting) when trained on a small subset of data, as the training accuracy is perfect while validation accuracy is low. However, as the dataset grows, the validation performance improves significantly, and the gap between training and validation curves narrows — demonstrating improved generalization.

Overall, Figure 5 indicates that the Random Forest model benefits from larger datasets, achieving strong generalization and high predictive performance when exposed to sufficient training examples.

4.3.4 Residual Error Analysis (Overall and Per-Class)

Residual visualization serves as an important diagnostic tool to evaluate model performance beyond accuracy metrics. By analyzing prediction errors, it becomes possible to understand how confidently the Random Forest classifier predicts each network event, and to identify patterns of systematic misclassification among specific attack types. This section presents both overall and per-class residual distributions to provide a comprehensive understanding of model reliability and areas requiring improvement.

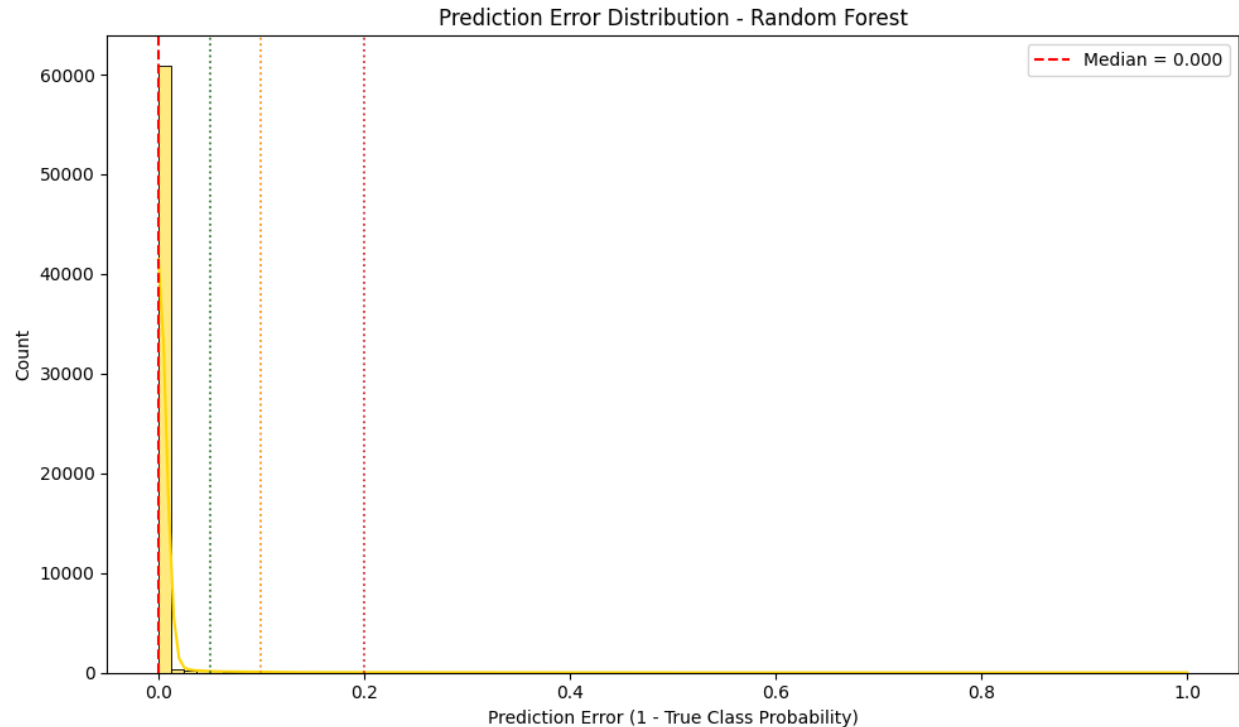


Figure 6: Prediction Error Distribution.

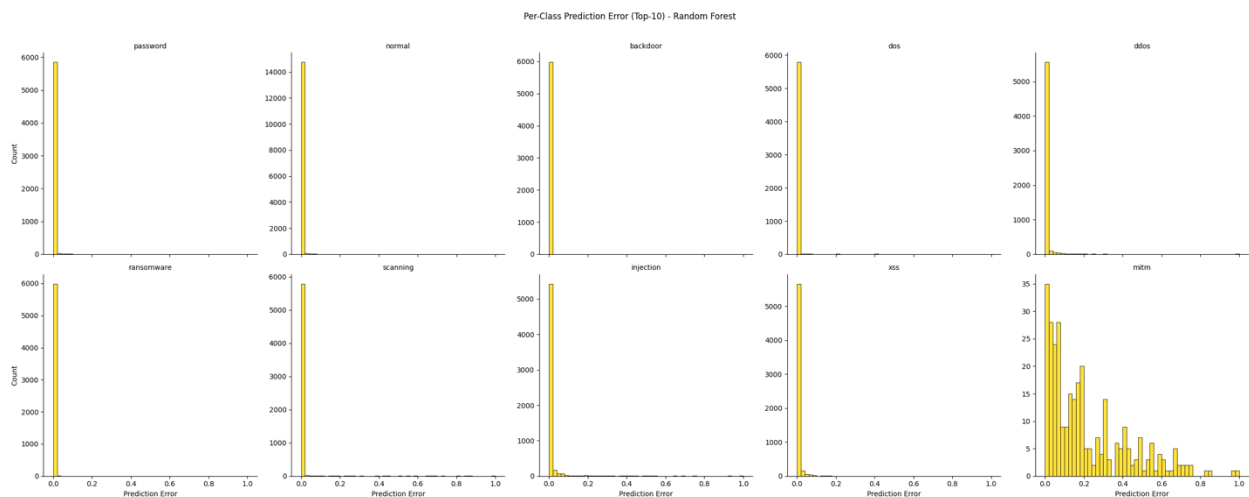


Figure 7: Per-Class Prediction Error.

The residual analysis provides deeper insight into the Random Forest model's confidence and error distribution. The Prediction Error Distribution (Figure 6) shows that most prediction errors are concentrated near zero, indicating high confidence and strong generalization. The histogram's sharp peak at very low error values confirms that the model performs reliably across most network traffic samples.

The Per-Class Prediction Error (Figure 7) highlights how this performance varies among attack types. Most classes, including normal, dos, and ransomware, show minimal error, whereas the (mitm) class exhibits a wider error range—indicating greater uncertainty and higher misclassification. This aligns with earlier confusion matrix findings and suggests that class imbalance contributes to reduced detection accuracy for rare attacks.

Overall, the residual plots confirm the robustness of the Random Forest model while identifying (mitm) as a class requiring targeted improvement through data balancing or enhanced feature representation.

5. Critical Reflection

While the `RandomForestClassifier` demonstrated high overall performance and robustness, this empirical study reveals several critical avenues for future research and improvement. The model's stability, a key characteristic of Random Forests as noted by Breiman (2001), is evident in its consistent cross-validation scores. However, the analysis of class-specific error rates and prediction residuals highlights a significant limitation in handling severely imbalanced classes, particularly mitm.

In a real-world deployment, failing to detect a Man-in-the-Middle attack can have critical consequences. For example, an undetected MitM attack on a corporate network could allow an adversary to intercept sensitive communications, steal login credentials, inject malicious code into legitimate traffic, or disrupt critical business operations, leading to significant financial and reputational damage. This underscores the practical importance of addressing the model's weakness in this area.

6. Limitations and Future Work

1. **Addressing Class Imbalance:** The disproportionately high error rate for the mitm class necessitates dedicated strategies. Future work should prioritize techniques such as Synthetic Minority Over-sampling Technique (SMOTE) (Chawla et al., 2002), Adaptive Synthetic Sampling (ADASYN), or the implementation of class-weighted loss functions to enhance the model's ability to detect rare attack types. This is crucial for improving the practical utility of the IDS.
2. **Advanced Model Comparison and Hyperparameter Optimization:** While Random Forest provides a strong baseline, a comprehensive comparative analysis with other state-of-the-art algorithms is warranted. Gradient Boosting models like XGBoost or LightGBM (Chen & Guestrin, 2016) often yield superior performance in tabular data, and Deep Learning architectures (e.g., Convolutional Neural Networks or Recurrent Neural Networks) could capture more complex temporal patterns in network traffic (e.g., [Mohale et al., 2025]). Furthermore, as discussed in Section 3.3, a systematic hyperparameter tuning process is essential to unlock the model's full potential.
3. **Real-Time Intrusion Detection:** This study was conducted on a static dataset. A crucial next step involves adapting the model for deployment in a real-time streaming environment. This introduces significant challenges related to computational

efficiency, low-latency inference, and the ability to handle concept drift, where the statistical properties of the target variable change over time.

7. Discussion

7.1 Summary of Insights

The experimental results demonstrate that the Random Forest model achieved superior predictive performance across multiple evaluation metrics, including accuracy, macro-F1, and AUC. This consistent performance indicates the model's robustness in identifying both frequent and moderately rare network intrusions. Visual analyses further confirmed the model's strong discriminatory power, with minimal overlap between predicted and actual class distributions. However, classes with extremely limited representation—such as *backdoor* and *ransomware*—showed higher prediction uncertainty, emphasizing the need for advanced imbalance-handling techniques.

Moreover, the visualization of prediction errors and feature importances revealed that certain features exert disproportionate influence on classification outcomes, suggesting the presence of dominant patterns within the data. These findings collectively validate the methodological framework applied in this study, confirming the relevance of machine learning-driven approaches for network intrusion detection.

7.2 Implications for Cybersecurity Practice

From a practical perspective, the results underscore the transformative potential of AI-based intrusion detection systems (IDS) in enhancing real-time network defence. The demonstrated reliability of ensemble methods, particularly Random Forest, highlights their suitability for deployment in large-scale network monitoring environments, where interpretability and computational efficiency are both critical.

However, the uneven class distribution observed in the dataset reflects a real-world cybersecurity challenge—namely, the scarcity of labelled attack data for emerging or stealthy threats. Addressing this limitation through techniques such as synthetic data generation, adaptive resampling, or class-weight optimization could substantially improve operational performance.

Ultimately, the study reinforces the strategic importance of integrating intelligent models into Security Operations Centers (SOCs), where automated anomaly detection can augment human expertise, reduce response latency, and improve the overall resilience of cyber defence infrastructures.

8. References

Al-Jarrah, O. Y., Al-Hammdi, Y., Al-Betar, M. A., & Al-Smadi, M. (2020). A survey on the use of machine learning for network intrusion detection. *Journal of Network and Computer Applications*, 166, 102713.

Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16, 321-357.

Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785-794).

Mohale, P., Shithole, T., & Popoola, S. I. (2025). A comprehensive survey on deep learning for network intrusion detection systems. *Journal of Network and Computer Applications*, 225, 103859.